Bloc A · El protocol

1. Per què MQTT fa servir un intermediari en lloc que els dispositius es parlin directament? Enumereu tres conseqüències d'aquesta decisió, i que almenys una sigui un inconvenient.
El broker desacobla els dispositius en temps, espai i sincronització. Els nodes no necessiten estar connectats simultàniament per intercanviar informació.

- Això fa que es simplifiqui enormement l'arquitectura, ja que afegir nous subscriptors o publicadors no requereix reconfigurar la resta de nodes de la xarxa.
- Es redueix el consum de memòria, processament i bateria
- El problema que te es que si el broker cau, tota la comunicació del sistema queda interrompuda.

2. Expliqueu els tres nivells de QoS i digueu, per a cadascun, quantes vegades pot arribar un missatge al subscriptor en el pitjor cas.
QoS 0: El missatge s'envia sense cap confirmació (es pot perdre per el camí)
QoS 1: El missatge s'envia i es reentrega fins que es rep un reconeixement
QoS 2: Garanteix el lliurament mitjançant un diàleg de 4 passos (PUBLISH, PUBREC, PUBREL, PUBCOMP).

3. Què és un missatge retingut i en què es diferencia d'un missatge amb QoS 1? Poseu un exemple del vostre projecte on el retingut sigui la solució correcta.
Un missatge retingut queda guardat a la memòria del broker com l'últim valor conegut d'un tema concret. Quan un nou subscriptor es connecta i s'hi subscriu, rep aquest missatge immediatament sense haver d'esperar que el publicador en generi un de nou. En canvi, QoS 1 només garanteix l'entrega durant el transport actiu del missatge i no guarda cap còpia per a futurs subscriptors.

En el nostre projecte seria ideal per l'estat de connexió del node (online/offline). Ja que ho volem saber immediatament quan obrim el Node-RED sense haber d'esperar a la següent lectura de telemetria.

4. Expliqueu el Last Will and Testament. Quant triga a disparar-se si el keepalive és de 60 segons?
El LWT és un missatge que el client registra al broker en el moment de connectar-se. Si el client es desconnecta de forma anòmala (pèrduda de cobertura, tall de potència…), el broker publica automàticament aquest missatge al tema especificat per alertar la resta del sistema.

Es dispara quan el broker detecta que ha passat el temps limit de desconnexió, Amb un keepalive de 60 segons, trigarà 90 segons (1,5 x 60 s) des del darrer missatge o control rebut.

5. Què vol dir una sessió persistent i què passa amb els missatges d'un client desconnectat amb clean_session a fals?
La sessió persisten vol dir que el broker manté la informació sobre el client (subscripcions actives, missatges QoS 1/2 pendents d'entrega) inclús quan aquest es desconnecta.

clean_session = false: Quan el client es desconnecta, el broker emmagatzema tots els missatges en cua (amb QoS > 0) enviats als temes als quals estava subscrit. En reconnectar-se el client amb el mateix ClientID, el broker li lliura automàticament tots els missatges acumulats durant la seva absència.

6. Un tema pot començar per /? Què implica? I es pot fer servir el comodí # enmig d'un tema?
Si que es pot començar amb /, però crea un nivell buit al principi. Exemple: /uvic/node1 té 3 nivells: "", uvic, node1. Això afegeix complexitat innecessària.

El # enmig d'un tema no es pot, només pot ocupar l'ultim nivell uvic/node1/#

Bloc B · Disseny del vostre sistema

1. Dissenyeu l'espai de noms de temes del vostre equip. Ha de contemplar: telemetria, estat del node, ordres cap al node i acusaments. Escriviu-lo com una llista de patrons amb les parts variables marcades.
Prefix: uvic/eric/
Telemetria: uvic/eric/{node_id}/telemetry/{sensor} (p. ex. uvic/eric/esp32-01/telemetry/temperature)
Estat del node: uvic/eric/{node_id}/status
Ordres cap al node: uvic/eric/{node_id}/cmd/{actuator}
Acusaments (Acks): uvic/eric/{node_id}/ack/{actuator}

2. Justifiqueu una decisió concreta: poseu l'identificador del node abans o després del tipus de dada? Què es pot fer fàcilment amb cada opció i què es fa difícil?
El poso abans del tipus de dada perquè aixi es més fàcil subscriurem a tots els missatges d'un sol node mitjançant el comodí uvic/eric/esp32-01/#

Però si volguessim rebre la temperatura de tots els nodes de la instal·lació d'un sol cop, caldria fer servir el comodí d'un sol nivell + (p. ex. uvic/eric/+/telemetry/temperature).

3. Dissenyeu el format del missatge de telemetria en JSON. Ha de portar, com a mínim, valor, unitat i marca de temps. Justifiqueu si hi poseu també l'identificador del node, tenint en compte que ja és al tema.
```
{
  "value": 23.5,
  "unit": "ºC",
  "timestamp": 1700000000
}
```
No és necessari incloure-hi l'identificador del node. Com que la informació ja està present en la jerarquia del tema MQTT al qual es publica

4. Digueu quins dels vostres temes han de ser retinguts i quins no, un per un.

Tema/Patró	Restringit?	Justificació
.../{node_id}/status	Sí	Cal conèixer l'últim estat del node immediatament en obrir el sistema.
.../{node_id}/telemetry/{sensor}	No	Les dades sensoritzades són pèrdues de temps; no té sentit rebre una lectura antiga en reconnectar-se.
.../{node_id}/cmd/{actuator}	No	Evita que un node reaccioni i executi una ordre antiga enviada en el passat al reconnectar-se.
.../{node_id}/ack/{actuator}	No	És una confirmació puntual d'un esdeveniment concret que ja ha passat.

5. Estimeu el trànsit: quants missatges per minut i quants bytes generarà el vostre sistema amb dos nodes?
Cada node envia 1 missatge de telemetria cada 10 segons (6 missatges/min per node).
Mida mitjana del missatge JSON + capçaleres: ~150 bytes.

$$2nodes \times 6msg/min = 12missatges/minut.$$

$$12msg/min \times 150bytes/msg = 1.800bytes/minut$$

Bloc C · El broker com a servei del sistema

1. Expliqueu, amb les vostres paraules, la diferència entre el paquet que instal·la apt, el servei que gestiona systemd i els fitxers de configuració i de dades que fa servir Mosquitto.
Paquet apt: És el conjunt de fitxers binaris executables i llibreries instal·lats al sistema operatiu (el programa Mosquitto en si).

Servei systemd: És el gestor encarregat d'executar l'executable de Mosquitto en segon pla, controlar-ne el cicle de vida (arrancada automàtica, reinici si cau, parades) i gestionar els permisos del procés.

Fitxers de configuració i dades: Els fitxers textuals (com mosquitto.conf) defineixen el comportament (ports, seguretat, permisos), mentre que la base de dades persistents (mosquitto.db) emmagatzema l'estat dels missatges retinguts i subscripcions.

2. Per què la configuració pròpia de Mosquitto es posa en un fitxer dins de etc/mosquitto/conf.d/ i no s'edita directament el /etc/mosquitto/mosquitto.conf que ve amb el paquet?
Quan el paquet de Mosquitto s'actualitza mitjançant l'eina apt, el fitxer /etc/mosquitto/mosquitto.conf principal pot ser sobreescrit. Utilitzant fitxers .conf separats dins del directori conf.d/, evitem perdre les nostres opcions personalitzades durant les actualitzacions del sistema.

3. Què fan systemctl enable i systemctl start i en què es diferencien? Què passa amb el broker quan el Raspberry Pi es reinicia si només heu fet una de les dues?
systemctl start mosquitto: Arranca el servei immediatament en la sessió actual.

systemctl enable mosquitto: Configura el servei per arrancar de forma automàtica cada vegada que el sistema operatiu (Raspberry Pi) s'inicïi.

Si només s'ha fet start, en reiniciar la Raspberry Pi el broker estarà aturat.
Si només s'ha fet enable, el broker s'engegarà automàticament després de reiniciar, tot i que no estigués en marcha abans del reinici.

4. Si el listener del port 1883 escolta a totes les interfícies (0.0.0.0), qui hi pot accedir des de la xarxa del laboratori? I si només escolta a localhost?
Si escolta a 0.0.0.0 qualsevol dispositiu o ordinador connectat a la xarxa local del laboratori pot connectar-se al broker.

Si només escolta a localhost (127.0.0.1) només els processos interns de la pròpia Raspberry Pi (com per exemple Node-RED executant-se de forma local) hi podran accedir; cap dispositiu extern de la xarxa (ni l'ESP32 ni el portàtil) hi podran connectar.

Exercici 4

Instalar Mosquitto a la Raspberry Pi:
```
# 1. Actualitzem
sudo apt update

# 2. Instal·lar mosquitto i les utilitats de client (mosquitto-clients)
sudo apt install -y mosquitto mosquitto-clients
```

Activar-ho com a servei amb systemctl:
```
# 1. Habilitar el servei perquè s'inicïi automàticament al reiniciar la Raspberry Pi
sudo systemctl enable mosquitto

# 2. Iniciar el servei immediatament en la sessió actual
sudo systemctl start mosquitto

# 3. Comprovar que el servei està actiu i executant-se correctament
sudo systemctl status mosquitto
```

Fitxer de configuració mínim:

```
# Escolta de connexions al port 1883 en totes les interfícies de xarxa
listener 1883 0.0.0.0

# Permetre connexions anònimes per a entorns de proves locals
allow_anonymous true

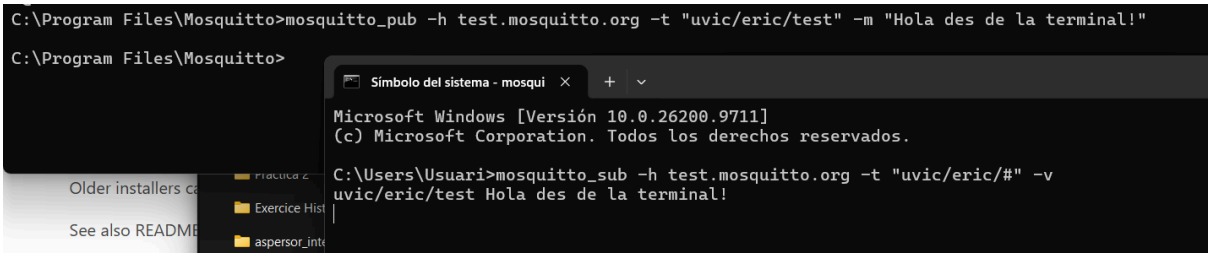
# Activar la persistència de dades (missatges retinguts i subscripcions)
persistence true
persistence_location /var/lib/mosquitto/
```

Subscriure's

mosquitto_sub -h test.mosquitto.org -t "uvic/eric/#" -v

Publicar un missatge

mosquitto_pub -h test.mosquitto.org -t "uvic/eric/test" -m "Hola des de la terminal!"



Fent servir Node-RED apuntant al broker podem veure que es rep tot el que s'envia.

